

Refactoring CRUD Application with **MVC Architecture** and **Blade Templates**

In this workshop, you will refactor your existing **Student List CRUD application (from Week 6)** to follow the MVC architectural pattern and implement the Blade templating engine.

PART 1: Setup

1. Create a new project inside your **htdocs** folder. Follow the given directory structure:

```
workshop8/
├── app/
│   ├── controllers/
│   ├── models/
│   └── views/
│       └── layouts/
├── public/
├── cache/
│   └── views/
```

2. Place your database connection file (**db.php**) in the project root directory (Inside **workshop8** folder). Use same code as workshop 6 (connection to same **school_db** database as done in workshop 6)

PART 2: Using Template Engine

Install Composer : <https://getcomposer.org/download/>

Use PHP installed in c:/xampp/php/php.exe (for Windows)

1. Open your terminal/command prompt and navigate to your project root directory (Inside workshop8 folder).

2. Initialize Composer using this command:

```
composer init
```

Follow the prompts, keep all settings as default

3. Install the Blade template engine using Composer:

```
composer require jenssegers/blade
```

Your folder structure should now look like this. All the composer and template specific files are added inside the vendor folder.

FOLDERS

- ▼  workshop8
 - ▼  app
 - ▶  controllers
 - ▶  models
 - ▶  views
 - ▼  cache
 - ▶  views
 - ▼  public
 - ▼  src
 - ▼  vendor
 - ▶  bin
 - ▶  carbonphp
 - ▶  composer
 - ▶  doctrine
 - ▶  illuminate
 - ▶  jenssegers
 - ▶  nesbot
 - ▶  psr
 - ▶  symfony
 - ▶  voku
 -  autoload.php
 - /* composer.json
 -  composer.lock
 -  db.php

PART 3: Creating the Model

1. Create a file **app/models/Student.php** that will contain the Student class.
2. Implement methods for all CRUD operations:
 - a. **all()** to retrieve all students
 - b. **find(\$id)** to retrieve a single student by ID
 - c. **create(\$name, \$email, \$course)** to insert a new student
 - d. **update(\$id, \$name, \$email, \$course)** to update an existing student
 - e. **delete(\$id)** to remove a student record

```
<?php

class Student{

    private PDO $pdo;

    public function __construct(PDO $pdo){ ...
    }

    public function all(){ ...
    }

    public function find(int $id){ ...
    }

    public function create(string $name, string $email, string $course){ ...
    }

    public function update(int $id, string $name, string $email, string $course){ ...
    }

    public function delete(int $id){ ...
    }

}
```

?>

PART 4: Creating the Controller

1. Create a file **app/controllers/StudentController.php** that will contain the StudentController class.
2. Implement the following **methods** in your controller:
 - a. **index()** to display all students
 - b. **create()** to show the form for adding a new student
 - c. **store()** to process the form submission for adding a student
 - d. **edit(\$id)** to show the form for editing an existing student
 - e. **update(\$id)** to process the form submission for updating a student
 - f. **delete(\$id)** to remove a student record

PART 5: Setting Up Blade and Routing

1. Create the main entry point file **public/index.php**.
2. Set up the **Blade engine** by specifying the **paths** to your views and cache directories.

Eg:

```
<?php

require __DIR__ . '/../vendor/autoload.php';

use Jenssegers\Blade\Blade;

$views = __DIR__ . '/../app/views';
$cache = __DIR__ . '/../cache/views';

$blade = new Blade($views, $cache);

?>
```

3. Implement a simple routing system that:

- a. **Reads** the requested page from the **URL parameters**
- b. **Instantiates** the **StudentController** with **dependencies**
- c. **Calls** the appropriate **controller** method based on the requested page

4. If you get version error while navigating to index.php through localhost

- a. Open **composer.json** file that's located in the root directory, then make changes to the versions of php and blade so that they align with each other.

You can use these recommended versions.

```
{
    "require": {
        "php": ">=8.0",
        "jenssegers/blade": "^1.4"
    }
}
```

- b. Update composer using terminal in your root directory. Run:

```
composer update
```

PART 6: Creating Blade Views

1. Create a master layout file **app/views/layouts/master.blade.php** that contains:
 - a. HTML structure (head, body)
 - b. CSS styling for your application
 - c. **@yield('content')** directive to define where child views will inject content
2. Create the following views that extend the master layout:
 - a. **app/views/students/index.blade.php**: Student list with actions (edit/delete) and a link to add a new student
 - b. **app/views/students/create.blade.php**: Form to add a new student
 - c. **app/views/students/edit.blade.php**: Form to edit an existing student
3. Use Blade syntax features:
 - a. **@extends** to inherit from the master layout
 - b. **@section** and **@endsection** to define content sections
 - c. **{{ }}** to display data
 - d. **@foreach** for looping through student records
 - e. **@if**, **@else**, **@endif** for conditional logic